机器学习系统 2026春

第四章 Transformer和大语言模型

张燕咏 讲席教授 yanyongz@ustc.edu.cn
张午阳 特任教授 wuyangz@ustc.edu.cn

中国科学技术大学
University of Science and Technology of China

一条ChatGPT回复背后的计算量

为什么: 理解Transformer架构, 才能理解这些巨大的数字从何而来

GPT-4 单次推理

~数千亿 FLOPs（MoE架构）

≈ H100 集群单次前向传播(prefill) ~数秒 (实际生成受限于显存带宽)

KV Cache (128K context)

单条请求即可达数十GB

batch增大后可能超过模型权重

这门课要回答的问题:

- Transformer为什么能取代RNN/CNN?
- 注意力机制的计算量从何而来?
- KV Cache为什么这么大? 怎么优化?
- Scaling Laws如何指导模型设计?
- 推理模型(o1/R1)带来了什么新挑战?

关键要点: Transformer是当前AI的基石架构，理解其系统特性, 是优化训练和推理的前提

课程路线图: 3节课 × 45分钟

第1节

从RNN到Transformer 架构全解析

- 为什么需要Transformer?
- 自注意力机制详解
- MHA, FFN, LayerNorm
- 位置编码
- 计算量分析
- PyTorch代码实现

第2节

设计演进 从原始Transformer到LLM

- Encoder/Decoder/Both
- RoPE旋转位置编码
- KV Cache内存分析
- MQA → GQA优化
- SwiGLU, RMSNorm
- Scaling Laws & LLM发展

第3节

LLM能力、推理模型 与系统挑战

- 涌现能力 & ICL
- RLHF → DPO对齐
- o1 & DeepSeek-R1推理模型
- 多模态LLM
- MoE稀疏计算
- Prefill vs Decode系统分析

关键要点: 从架构原理 → 设计演进 → 系统挑战, 三节课构建完整的Transformer知识体系

PART 01

为什么需要Transformer?

从RNN的系统瓶颈到注意力机制的诞生

自然语言处理任务总览

判别任务 (Discriminative)

- 情感分析: 这部电影好看吗? → 正面/负面
- 文本分类: 这是体育/科技/财经新闻?
- 文本蕴含: A能推出B吗?
- 核心: 输入序列 → 类别标签

生成任务 (Generative)

- 语言建模: 预测下一个词
- 机器翻译: 英文→中文
- 文本摘要: 长文→要点
- 核心: 输入序列 → 输出序列

系统视角: 判别任务需要编码整个序列(双向), 生成任务需要逐步解码(单向), 不同需求催生不同架构

关键要点: 判别和生成是两类核心NLP任务, 它们对模型架构提出不同的计算需求

RNN: 串行计算的系统代价

问题: RNN的隐状态必须串行计算, 无法利用GPU的并行能力

RNN计算链:

$$h_0 \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \ldots \rightarrow h$$

$$\uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow$$

$$x_1 \quad x_2 \quad x_3 \quad x_4 \quad x$$

- **串行依赖**
- $h_t = f(h_{t-1}, x_t)$, 状态t依赖状态t-1
- N个token需要N步顺序计算
- **长期依赖困难**
- 建模token i和j的交互需要 O(|i-j|) 步
- 梯度消失/爆炸 (LSTM部分缓解)

GPU利用率:

RNN训练

GPU利用率极低: 大量时间在等待前一步完成
数千个CUDA核心只有少数在工作

理想的并行模型

所有token同时处理
数千核心满载运行
训练速度提升数百倍

- 语言可能具有长距离依赖:
- "The cat, which was sitting on the mat that I bought last week, is sleeping."

关键要点: RNN的串行本质是其最大系统瓶颈, 无法利用GPU并行计算, 且难以建模长距离依赖

CNN的尝试与局限

√ 优势: 可并行

1D卷积沿序列滑动
token间无依赖 → 可完全并行
GPU利用率大幅提升

✗ 局限: 感受野有限

单层卷积核大小=k, 只能看k个token
捕捉距离d的关系需要 O(d/k) 层
长距离交互 → 需要非常深的网络

特性	RNN	CNN	理想方案 (?)
并行性	✗ O(n)串行	√ 完全并行	√ 完全并行
任意距离交互	O(n) 步	O(n/k) 层	O(1)
全局上下文	√ (理论上)	✗ 受限于层数	√
GPU利用率	低	高	高

关键要点: CNN解决了并行性, 但上下文窗口受限，我们需要一种同时具备并行性和全局上下文的机制

我们需要什么?

完全并行

所有token同时处理, 充分利用GPU

全局上下文

每个token直接看到所有其他token

O(1)任意距离

任意两个token的交互不需要中间步骤

自注意力机制 (Self-Attention)

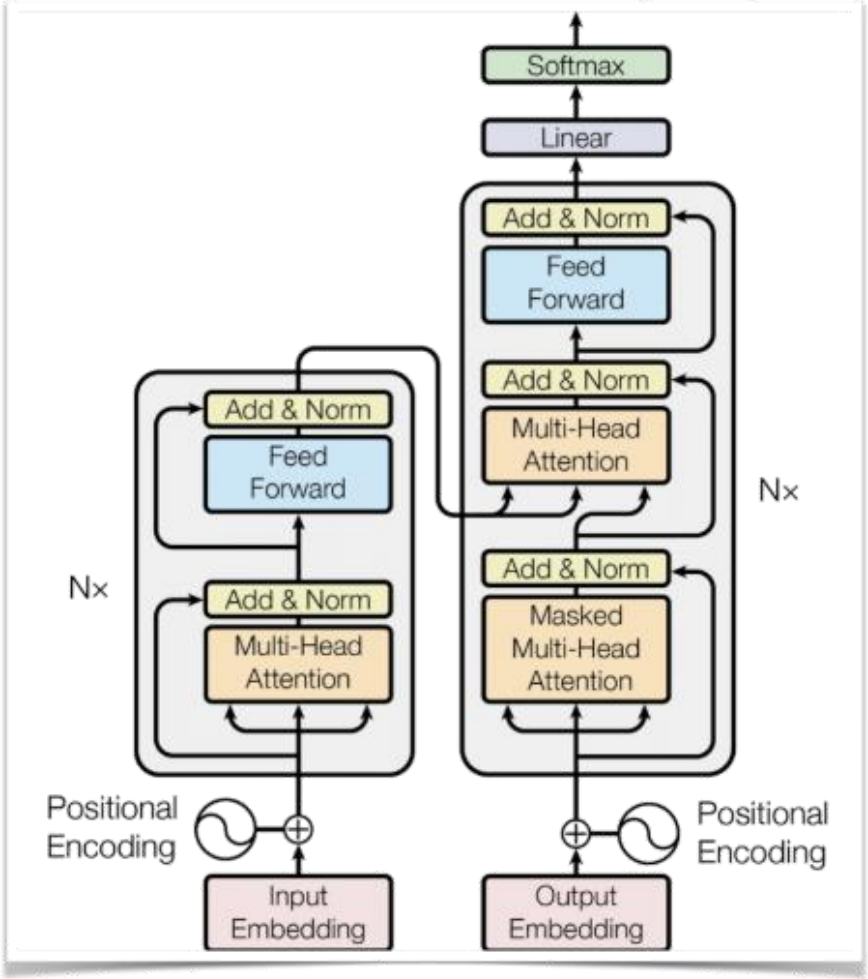
$$\text{Attention}(Q,K,V) = \text{Softmax}(QK^T/\sqrt{d}) \cdot V$$

关键要点: 自注意力同时满足三个需求: 并行计算 + 全局上下文 + O(1)任意距离交互, 代价是 $O(n^2)$ 复杂度

PART 02

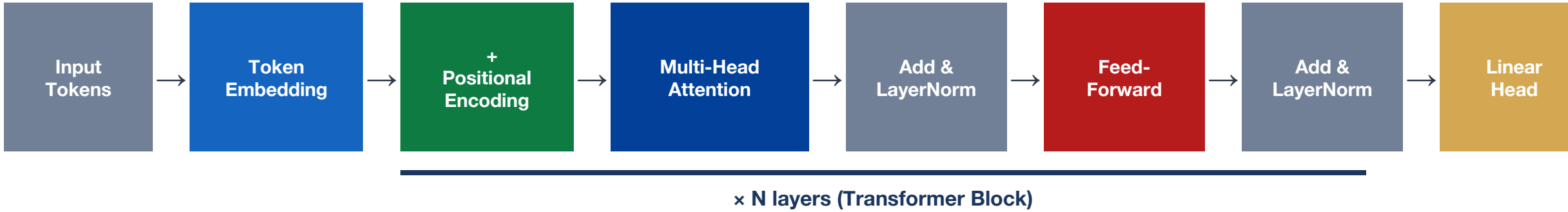
Transformer架构逐层拆解

从Tokenization到最终预测，理解每个组件的设计动机



Transformer全景架构

"Attention Is All You Need" (Vaswani et al., 2017)，以Decoder-only为例展示核心组件



Tokenization

单词→子词token (BPE)

Embedding

token→d维连续向量

Self-Attention

全局上下文建模 $O(n^2d)$

FFN

非线性特征变换，参数占~2/3

LayerNorm

训练稳定性保障

Residual

梯度直通，深层训练

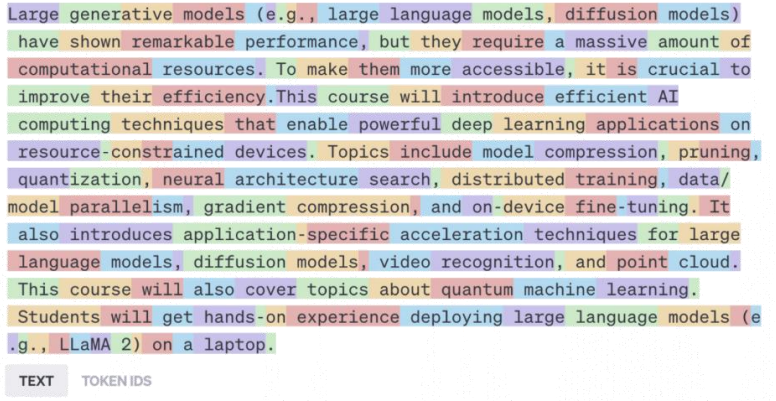
关键要点: Transformer由6个核心组件构成，接下来逐一拆解每个组件的设计动机和计算特性

Tokenization: 从单词到Token

为什么: 直接用单词作为输入? 词汇表太大(>100万) → 嵌入矩阵参数爆炸, 且无法处理新词

- **Byte Pair Encoding (BPE)**
- 统计语料中最频繁的字符对, 逐步合并
- "unhappiness" → ["un", "happi", "ness"]
- 110个单词 → 162个token (约1.5倍)

- **系统影响**
- Vocab size决定嵌入矩阵大小: V × d_model
- LLaMA: 32K vocab × 4096 dim = 128M参数
- GPT-4: ~100K vocab → 更大的嵌入层
- 更大vocab = 更短序列 = 稀疏token = 训练不充分




```
# Tokenizer示例 (tiktoken)
import
enc = tiktoken.get_encoding("cl100k_base")

text = "Transformer改变了NLP"
tokens = enc.encode(text)
# [Trans, former, 改, 变, 了, NLP]
# 6个中文字 → 可能8–10个token

print(f"Text: {len(text)} chars")
print(f"Tokens: {len(tokens)} tokens")
```

模型	Vocab Size	方法
GPT-2	50,257	BPE
LLaMA	32,000	SentencePiece
LLaMA-3	128,000	tiktoken BPE

关键要点: Tokenization决定了序列长度和嵌入参数量, vocab越大,序列越短,但嵌入层越大,是一个系统权衡

Word Embeddings: 从离散到连续

为什么: 神经网络需要连续数值输入, one-hot编码太稀疏 (V维向量, 只有1个非零)

One-Hot 编码

"cat" → [0, 0, 1, 0, ..., 0]（V维）

- 维度 = 词汇表大小 (32K~128K)
- 极度稀疏, 无法表达语义相似性
- "cat"和"kitten"的向量正交!

词嵌入 (Word Embedding)

"cat" → [0.21, −0.35, 0.78, ...]（d维）

- 维度 = d_model (典型512~4096)
- 稠密向量, 语义相似→向量接近
- 通过查表实现: E[token_id] → d维向量

嵌入矩阵 E ∈ ℝ^{V×d}

```
# PyTorch嵌入层
import torch.nn as nn

vocab_size = 32000
d_model = 4096

# 嵌入矩阵: 32000 × 4096
embed = nn.Embedding(vocab_size, d_model)
# 参数量: 32000 × 4096 = 131M

# token id → 向量
token_ids = torch.tensor([15, 892, 3])
vectors = embed(token_ids)
# shape: (3, 4096)
```

模型	V	d_model	嵌入参数量
LLaMA-7B	32K	4096	131M (1.9%)
LLaMA-3-8B	128K	4096	524M (6.5%)

关键要点: 嵌入层将离散token映射为连续向量，参数量 = V×d, vocab越大嵌入层越占比

自注意力: 直觉理解

核心思想: 每个token通过"提问"所有其他token来获取上下文信息

例: "The cat sat on the mat"

当处理 "sat" 时:
sat的Query → 和所有token的Key计算相似度

- 和当前 token 更相关的词，给更大权重
  不相关的词，给更小权重
  把相关词的信息聚合过来，更新当前 token 的表示

Attention权重 (示意):

The → 0.05（低关注）

cat → 0.45（高关注: 谁sat?）

sat → 0.15（自身）

on → 0.10

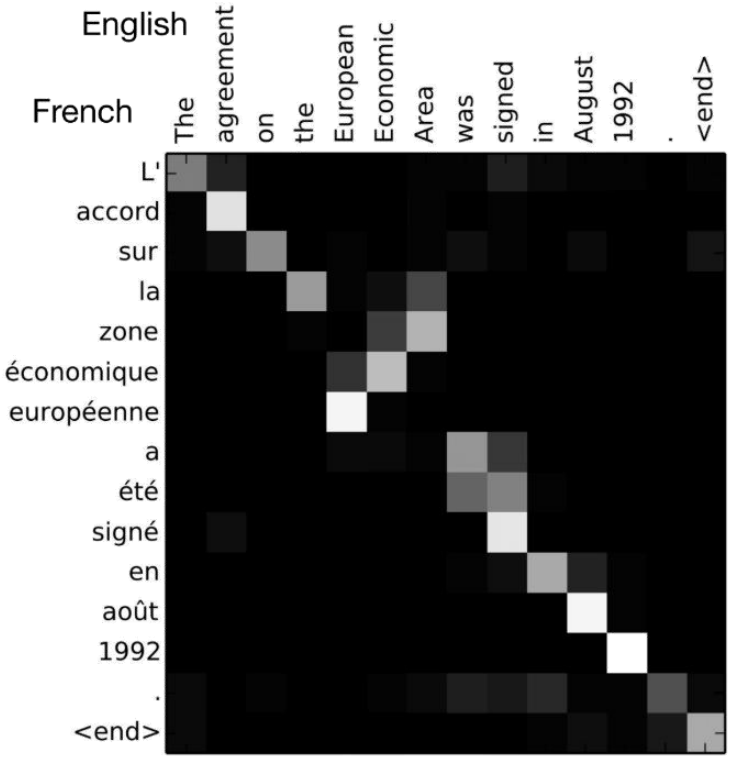
the → 0.05

mat → 0.20（关注: sat在哪?）

在机器翻译中, 注意力矩阵会自动学习词语对齐关系 (如 "猫"↔"cat")

- **三个角色: Query, Key, Value**
- Query (查询): "我在找什么信息?"
- Key (键): "我包含什么信息?"
- Value (值): "我能提供什么内容?"

- **计算过程:**
- 1. 每个token生成自己的Q, K, V向量
- 2. Q和所有K做点积 → 相关性分数
- 3. Softmax归一化 → 注意力权重
- 4. 用权重对V加权求和 → 上下文表示



关键要点: 自注意力让每个token"看到"所有其他token，通过Q·K相似度决定关注谁, 用V提取信息

自注意力: 数学与计算复杂度

公式推导:

输入: $X \in \mathbb{R}^{n \times d}$ (n个token, d维)

$Q = XWQ \quad K = XWK \quad V = XWV$

$WQ, WK, WV \in \mathbb{R}^{d \times d}$

$$\mathbf{Attention(Q,K,V) = Softmax(QK^T/\sqrt{d}) \cdot V}$$

- $QK^T \in \mathbb{R}^{n \times n}$, 注意力矩阵
- $/\sqrt{d}$, 缩放因子, 防止点积过大
- Softmax, 行归一化为概率分布
- $\cdot V$, 加权聚合value信息

为什么除以$\sqrt{d}$? 当d很大时, QK点积的方差≈d, Softmax会趋向one-hot, 梯度消失

计算复杂度分析:

操作	复杂度	说明
$X \rightarrow Q,K,V$	$O(nd^2)$	3个矩阵乘法
QK^T	$O(n^2d)$	注意力分数
Softmax·V	$O(n^2d)$	加权求和
总计	$O(n^2d + nd^2)$	n和d的竞争

内存瓶颈:

注意力矩阵 $QK^T \in \mathbb{R}^{n \times n}$

n=4096, fp16: $4096^2 \times 2B = 32MB/head$

32 heads → 1GB per layer!

→ FlashAttention的动机 (后续章节)

关键要点: 自注意力复杂度 $O(n^2d)$: 序列长度n翻倍 → 计算量4倍, 这是长序列推理的核心系统挑战

代码: Self-Attention in PyTorch

```
import torch
import torch.nn as nn
import math

class SelfAttention(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.d = d_model
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)

    def forward(self, x):          # x: (B, n, d)
        Q = self.W_q(x)            # ★ Query投影
        K = self.W_k(x)            # ★ Key投影
        V = self.W_v(x)            # ★ Value投影

        # ★ 注意力分数: (B, n, n)
        scores = Q @ K.transpose(-2, -1)
        scores = scores / math.sqrt(self.d)

        attn = torch.softmax(scores, dim=-1)
        out = attn @ V             # ★ 加权聚合
        return out
```

- **代码解读:**

- **L9-11: 三个线性投影**
- W_q, W_k, W_v: d×d矩阵
- 参数量: $3 \times d^2$ (主要开销)

- **L14-16: Q/K/V投影**
- 输入x经过线性变换
- 每个token生成自己的Q,K,V

- **L19-20: 注意力计算**
- $Q @ K^T \rightarrow n \times n$相似度矩阵
- 除以$\sqrt{d}$防止梯度消失

- **L22-23: Softmax + 聚合**
- 行归一化 → 注意力权重
- attn @ V → 上下文向量

关键要点: Self-Attention核心: 3个矩阵乘法(投影) + 1个矩阵乘法(QK^T) + Softmax + 1个矩阵乘法(·V)

多头注意力 (Multi-Head Attention)

为什么多头? 单个注意力头只能学一种关系模式，多个头可以同时捕捉不同的语义关系

- **单头 vs 多头**
- 单头: d_model → d_model (一种注意力模式)
- 多头: d_model → h个头, 每头 d_k = d_model/h

- **计算过程:**
- 1. 将Q,K,V按head维度分割
- $Q \in \mathbb{R}^{n \times d} \rightarrow$ h个 $Q_i \in \mathbb{R}^{n \times (d/h)}$
- 2. 每个head独立做attention
- head_i = Attention(Q_i, K_i, V_i)
- 3. 拼接所有head的输出
- MultiHead = Concat(head_1,...,head_h)·W_O

- **参数量不变!**
- 总投影矩阵大小仍是$3d^2 + d^2 = 4d^2$

不同头学到不同模式:

Head 1: 语法关系

主语-谓语对应

Head 2: 指代关系

代词-先行词

Head 3: 相邻关系

局部n-gram模式

Head 4: 长距离依赖

跨句关联

系统视角: MHA的h个head可以完全并行计算，很好地利用GPU的并行能力

关键要点: 多头注意力: 相同参数量, 多种注意力模式并行，h个头完美映射到GPU并行单元

Causal Mask: 生成任务的注意力约束

问题: 生成下一个词时, 模型不能"偷看"未来的词，需要屏蔽未来位置

- **为什么需要Mask?**
- 训练时: 所有位置同时计算 (并行)
- 但位置t的预测只能依赖 ≤t 的token
- 否则模型直接"复制"答案, 学不到东西

- **Causal Mask (因果掩码):**
- 在attention score上加 -∞ mask
- Softmax(-∞) = 0 → 完全屏蔽未来
- 形成下三角矩阵 (每行只看左边)

- **训练 vs 推理:**
- 训练: mask允许并行计算所有位置
- 推理: 逐token生成, mask天然满足

Causal Mask 矩阵:

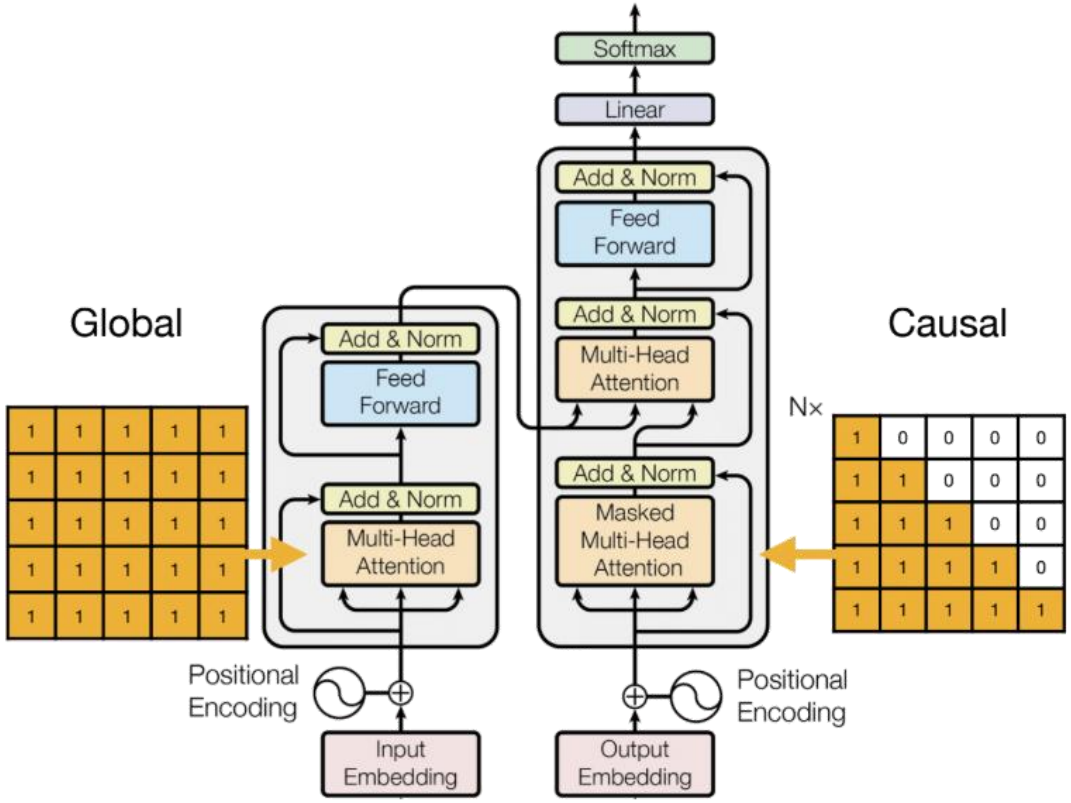
$$
\begin{array}{c|ccccc}
 & t_1 & t_2 & t_3 & t_4 & t_5 \\
\hline
t_1 & \surd & \times & \times & \times & \times \\
t_2 & \surd & \surd & \times & \times & \times \\
t_3 & \surd & \surd & \surd & \times & \times \\
t_4 & \surd & \surd & \surd & \surd & \times \\
t_5 & \surd & \surd & \surd & \surd & \surd
\end{array}
$$

√ = 可以看到 (mask=0)

✗ = 被屏蔽 (mask=-∞)

对比BERT (双向):

BERT不使用causal mask
每个token可以看到所有位置
适合理解任务, 不适合生成



关键要点: Causal mask让训练并行但预测单向，这是Decoder-only模型(GPT系列)的核心约束

前馈网络 (FFN): 隐藏的参数大户

为什么: 注意力只做线性加权，FFN提供元素级非线性变换，是模型"记忆"知识的主要载体

- **FFN结构: 两层MLP (反向瓶颈)**
- $FFN(x) = W_2 \cdot GELU(W_1x + b_1) + b_2$
- $W_1 \in \mathbb{R}^{d \times 4d}$，扩展4倍
- $W_2 \in \mathbb{R}^{4d \times d}$，压缩回来

- **为什么4倍扩展?**
- 更大的隐层 → 更强的表达能力
- 4倍是经验值，性能-效率平衡点
- SwiGLU用8/3倍扩展但有3个矩阵 (后续讨论)

参数量对比: FFN vs Attention

组件	参数量	占比
Attention (Q,K,V,O)	$4d^2$	~1/3
FFN (W_1, W_2)	**$8d^2$**	**~2/3**
LayerNorm	4d	≈0
Total per block	$12d^2$	100%

系统洞察:

FFN占了2/3的参数和计算

LLaMA-7B(SwiGLU): 每层FFN有 3×4096×11008 ≈ 135M参数

32层 → FFN总参数 ~4.3B (占7B的62%)

关键要点: FFN是Transformer的参数大户(~2/3参数)，它是模型存储世界知识的主要组件，也是MoE优化的目标

LayerNorm + 残差连接: 训练稳定性

为什么: 没有归一化和残差连接, 深层Transformer无法训练 (梯度消失/爆炸)

Layer Normalization

对每个token的嵌入向量独立归一化:

$$LN(x) = \gamma \cdot (x - \mu) / (\sigma + \varepsilon) + \beta$$

- μ, σ 沿embedding维度计算
- γ, β 是可学习参数 (2d个)
- 与BatchNorm不同: 不依赖batch统计

RMSNorm (现代选择):

$$RMSNorm(x) = x / RMS(x) \cdot \gamma$$

去掉均值偏移, 更高效 (LLaMA使用)

Pre-norm vs Post-norm

Post-norm (原始Transformer):

x → Attention → Add(x) → LN

- 训练不稳定, 需要warmup
- 深层模型容易发散

Pre-norm (现代标准):

x → LN → Attention → Add(x)

- 训练更稳定, 无需warmup
- LLaMA/GPT/Mistral均使用Pre-norm

残差连接: output = x + SubLayer(x) , 梯度直通, 允许信息跨层传递, 是训练深层网络的关键

关键要点: Pre-RMSNorm + 残差连接 = 现代Transformer训练的标配, 保证100+层网络稳定训练

位置编码: 让Transformer感知顺序

问题: 自注意力是集合操作，"cat sat mat"和"mat sat cat"的计算结果完全相同!

- **为什么需要位置信息?**
- Attention(Q,K,V)中, 打乱输入顺序不影响输出
- "I eat burger" = " burger eat I" (对attention而言)
- 必须显式注入位置信息

- **正弦位置编码 (原始Transformer)**
- PE(pos, 2i) = sin(pos / 10000^(2i/d))
- PE(pos, 2i+1) = cos(pos / 10000^(2i/d))

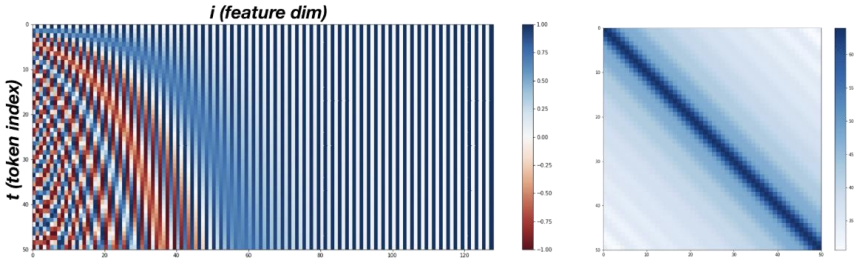
- 直觉: 不同频率的正弦波编码不同尺度的位置
- 加到嵌入向量上: input = embedding + PE

位置编码类型对比:

类型	方法	代表模型
绝对位置	加到输入embedding	原始Transformer
可学习绝对	学习位置embedding	GPT-1/2, BERT
相对位置	修改attention score	T5, ALiBi
旋转位置	**旋转Q/K向量**	**LLaMA, Mistral**

- **绝对编码的问题:**
- 训练时序列长度固定 (如2048)
- 推理时遇到更长序列 → 位置编码越界
- 相对编码 (RoPE/ALiBi) 可更好泛化
- → 第二节课详细讲解RoPE

$$\vec{p}_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k . t), & \text{if } i = 2k \\ \cos(\omega_k . t), & \text{if } i = 2k+1 \end{cases} \qquad \omega_k = \frac{1}{10000^{2k/d}}$$



关键要点: 位置编码是Transformer感知序列顺序的唯一方式，现代LLM已从绝对编码演进到旋转位置编码(RoPE)

代码: Transformer Block in PyTorch

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.norm1 = nn.RMSNorm(d_model)     # ★ Pre-norm
        self.attn = MultiHeadAttn(d_model, n_heads)
        self.norm2 = nn.RMSNorm(d_model)
        self.ffn = FFN(d_model)

    def forward(self, x, mask=None):
        # ★ Pre-norm + Residual pattern
        h = self.norm1(x)
        h = self.attn(h, mask=mask)
        x = x + h              # ★ 残差连接

        h = self.norm2(x)
        h = self.ffn(h)
        x = x + h              # ★ 残差连接
        return x

class FFN(nn.Module):
    def __init__(self, d, mult=4):
        super().__init__()
        hidden = d * mult
        self.w1 = nn.Linear(d, hidden)
        self.w2 = nn.Linear(hidden, d)
```

- **数据流 (Pre-norm):**

```
x
│
├→ RMSNorm → MHA → +
│                  │ (残差)
├──────────────────────────┘
│
├→ RMSNorm → FFN → +
│                  │ (残差)
├──────────────────────────┘
x' (output)
```

- **整个Transformer:**
- = N个Block堆叠
- LLaMA-7B: N=32
- LLaMA-70B: N=80

关键要点: 一个Transformer Block = RMSNorm→MHA→Residual + RMSNorm→FFN→Residual, 堆叠N次即为完整模型

Transformer计算量全景拆解

系统核心: 理解FLOPs分布, 才能知道优化哪个组件收益最大

操作	FLOPs	说明	n=2K, d=4096 FLOPs
QKV投影	$6nd^2$	3个 (n×d)×(d×d) 矩阵乘	~201G
Attention Score (QK^T)	$2n^2d$	n×d 乘 d×n	~34G
Attention·V	$2n^2d$	n×n 乘 n×d	~34G
Output投影	$2nd^2$	(n×d)×(d×d)	~67G
FFN (W_1+W_2)	**$16nd^2$**	**两个 d×4d 矩阵乘**	**~537G**
Total per block	**$24nd^2 + 4n^2d$**		**~873G**

n 较小时 (n << d):

$24nd^2$ 主导 → FFN是计算瓶颈

n=2K, d=4096: FFN占 ~62%

n 较大时 (n >> d):

$4n^2d$ 主导 → Attention是计算瓶颈

n=128K, d=4096: Attention占 ~84%

关键要点: 短序列优化FFN (MoE), 长序列优化Attention (FlashAttention, 稀疏注意力), 瓶颈随序列长度变化

原始Transformer: 结果与影响

机器翻译基准 (WMT 2014):

模型	EN→DE	EN→FR	训练成本
GNMT (RNN)	26.03	39.92	大
ConvS2S (CNN)	25.16	40.46	大
Transformer	**28.4**	**41.0**	**1/4**

- **关键突破:**
- BLEU分数超越所有已有模型
- 训练成本仅为之前最佳的1/4
- 并行训练 = 更多数据 + 更大模型

Transformer的深远影响:

- **2017** Attention Is All You Need
- **2018** BERT: NLP理解任务全面突破
- **2018** GPT-1: 自回归预训练
- **2020** GPT-3: 175B, 涌现In-Context Learning
- **2020** ViT: Transformer进入计算机视觉
- **2022** ChatGPT: AI进入公众视野
- **2023** GPT-4, LLaMA, 开源LLM时代
- **2024** o1, Reasoning Models, 多模态

关键要点: Transformer用1/4训练成本超越所有模型，并行化不只是快, 更意味着可以训练更大的模型

课堂思考 & 第一节课总结

课堂思考题:

1. 如果序列长度n翻倍, Transformer的计算量增加多少? 内存呢?
2. 为什么FFN要用4倍扩展而不是2倍或8倍?
3. 如果去掉残差连接, 一个32层Transformer还能训练吗?

并行 > 串行

RNN串行→GPU浪费
Transformer并行→规模化基础

$O(n^2d)$ 复杂度

短序列FFN主导
长序列Attention主导

6个核心组件

Token+Embed+MHA+FFN
+LayerNorm+PositionalEnc

关键要点: 下节课: 每个设计改进背后都有一个系统瓶颈， RoPE, GQA, SwiGLU, KV Cache优化

70B推理: KV缓存比模型权重还大!

为什么: 原始Transformer的每个设计选择都面临系统瓶颈，改进都是被"逼"出来的

KV Cache Memory (MHA)

~160 GB

在标准 MHA、FP16、80 层、hidden size 约 8192 的 70B 级模型, bs=16, seq_len=4096

仅KV缓存就需两块A100! (还未算权重~140GB)

本节课要回答:

- Encoder-only vs Decoder-only: 为什么LLM选了后者?
- 绝对位置 → RoPE: 怎么让模型泛化到更长序列?
- MHA → GQA: 怎么把KV缓存缩小8倍?
- FFN → SwiGLU: 简单改动为什么有效?
- Scaling Laws: 固定算力, 模型该多大?

关键要点: 每个设计改进都源于系统瓶颈: 内存、计算、泛化，理解"为什么改"比"改了什么"更重要

PART 03

架构三条路线

Encoder-Decoder, Encoder-only, Decoder-only: 殊途同归?

三种Transformer架构对比

特性	Encoder-Decoder (T5)	Encoder-only (BERT)	Decoder-only (GPT)
注意力	Encoder双向+ Decoder因果+交叉	双向 (全连接)	因果 (下三角)
预训练目标	跨度掩码去噪 (denoising)	Masked LM (完形填空)	Next token prediction
典型任务	翻译, 摘要 (输入→输出)	分类, NER (理解)	生成, 对话 (创造)
代表模型	T5, BART Flan-T5	BERT, RoBERTa DeBERTa	GPT-3/4, LLaMA Mistral, Claude
规模化	结构复杂 难以极大规模化	不能生成 应用场景受限	**简单统一** **最易规模化**

关键要点: 三种架构各有所长, 但Decoder-only凭借统一框架和规模化优势, 成为LLM时代的主流选择

BERT: 双向编码器

核心创新: 用Masked Language Model预训练, 让每个token"看到"上下文双向信息

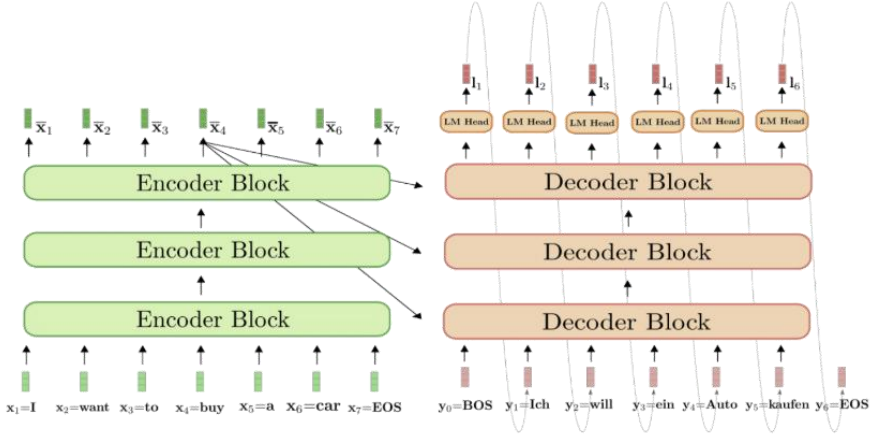
- **预训练任务 #1: Masked LM (MLM)**
- 随机mask 15%的token
- 训练模型预测被mask的词
- "The [MASK] sat on the mat" → cat

- **预训练任务 #2: Next Sentence Prediction**
- 判断句子B是否是A的下一句
- (后续研究发现作用不大)

- **使用方式: 预训练 + 微调**
- 在大语料上预训练
- 在下游任务上加分类头微调
- BERT-Base: 110M, BERT-Large: 340M

Attention Mask (双向):

	t_1	t_2	t_3	t_4	t_5
t_1	√	√	√	√	√
t_2	√	√	√	√	√
t_3	√	√	√	√	√
t_4	√	√	√	√	√
t_5	√	√	√	√	√



每个token可以看到所有其他token → 适合理解任务

局限:

- 不能自然地做生成任务
- 训练只用15% token的损失 (低效)
- 规模化困难, 最大仅~1B参数

关键要点: BERT用双向注意力+MLM实现强大的语言理解, 但无法生成文本, 且规模化受限

GPT: 自回归解码器

核心创新: 用Next Token Prediction预训练，简单、统一、可无限规模化

- **预训练: Next Token Prediction**
- 给定前面所有token, 预测下一个
- $P(x \mid x_1, x_2, \ldots, x_{-1})$
- 损失函数: 交叉熵, 每个token都参与

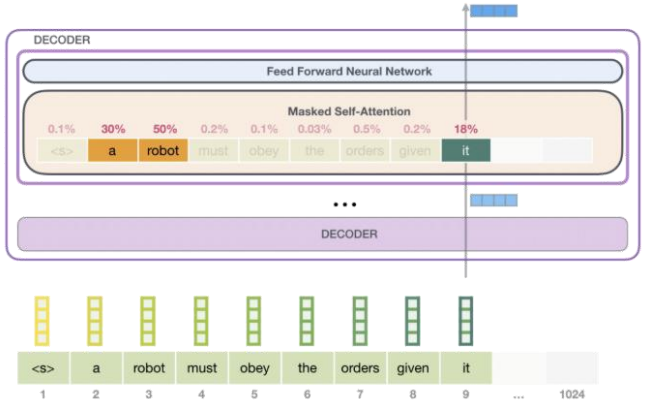
- **为什么高效?**
- 100% token利用率 (vs BERT的15%)
- 每个序列提供n个训练样本
- 天然适合生成任务

- **使用方式的演进:**
- GPT-1 (117M): 预训练+微调
- GPT-2 (1.5B): 零样本开始可行
- GPT-3 (175B): In-Context Learning!

自回归生成过程:

```
# 自回归生成 (贪心)
prompt = "The capital of France"
tokens = tokenize(prompt)

for i in range(max_len):
    logits = model(tokens)
    next_token = logits[-1].argmax()
    tokens.append(next_token)
    if next_token == EOS:
        break
# → "The capital of France is Paris"
```



优势 (系统视角):

- 架构最简单: 只有decoder, 无cross-attn
- 训练最高效: 每个token都计算loss
- 规模化最好: 从117M到1.8T参数

关键要点: GPT式Decoder-only: 最简单的架构, 最高效的训练, 最好的规模化特性，大道至简

为什么LLM时代选择了Decoder-only?

统一框架

一切皆生成:
分类? 生成"正面/负面"
翻译? 生成目标语言
摘要? 生成简短版本
无需为每个任务设计结构

训练效率

每个token都有loss信号:
BERT: 15% token有梯度
GPT: 100% token有梯度
同样数据, GPT学到更多

规模化收益

架构越简单,规模越大:
无encoder-decoder对齐
参数全部用于单一模型

关键要点: Decoder-only = 最简单 × 最高效 × 最易规模化，2023年后几乎所有主流LLM都采用这一架构

PART 04

关键设计改进

位置编码、KV缓存、FFN，每个改进背后的系统动机

位置编码进化: 从绝对到相对

问题: 绝对位置编码无法泛化到训练时未见过的序列长度，训练2K, 推理时4K怎么办?

绝对位置编码的问题:

- 位置信息加到输入embedding
- pos=2049时, 编码从未在训练中出现
- 可学习位置: 超出范围直接越界
- 正弦位置: 理论可外推, 实际效果差

相对位置编码的优势:

- 只关心两个token间的相对距离
- 不依赖绝对索引, 天然支持更长序列
- 作用于attention score而非输入
- 实现: ALiBi, RoPE (当前主流)

对比总结:

	绝对PE	ALiBi	RoPE
作用位置	输入层	Attention	Q和K
可学习?	可选	否	否
长度外推	差	较好	好(+NTK)
额外参数	n×d或0	0	0
额外计算	加法	加法	旋转乘法
使用率	少	少	主流

几乎所有现代LLM (LLaMA, Mistral, Qwen, DeepSeek) 都使用RoPE

关键要点: 相对位置编码解决了长度泛化问题，RoPE已成为事实标准, 支撑LLM从2K扩展到128K+上下文

ALiBi: 简单的线性偏置

思路: 不在输入上加位置, 而在attention score上加距离相关的偏置

- **Attention with Linear Biases**
- Softmax($QK^T/\sqrt{d}$ + m · bias)
- bias[i,j] = -|i - j| (距离越远, 惩罚越大)
- **m: 每个head不同的斜率**
- m = 2^(-8/h), 2^(-16/h), ...
- 不同head关注不同距离范围
- 小斜率head → 关注远距离
- 大斜率head → 关注近距离

Bias矩阵示例 (m=1):

$$
\begin{array}{c|cccc}
 & t_1 & t_2 & t_3 & t_4 \\
\hline
t_1 & 0 & -\infty & -\infty & -\infty \\
t_2 & -1 & 0 & -\infty & -\infty \\
t_3 & -2 & -1 & 0 & -\infty \\
t_4 & -3 & -2 & -1 & 0
\end{array}
$$

(结合causal mask, 未来位置=$-\infty$)

特点:

√ 无额外参数, 无额外学习

√ 实现极简 (几行代码)

✗ 长度外推效果不如RoPE+NTK

关键要点: ALiBi用最简单的方式引入位置信息, 距离越远惩罚越大, 但RoPE的效果和灵活性更优

RoPE: 旋转位置编码

核心直觉: 用旋转来编码位置，两个向量的相对旋转角度只取决于它们的相对位置

- **旋转的直觉 (2D)**
- 把embedding的每2个维度看成一个2D平面
- 位置m的token → 旋转 m·θ 角度

- Q_m和K_n做点积时:
- 点积只取决于旋转角度差 (m-n)·θ
- → 天然编码相对位置!

- **公式:**
- $f(x, m) = R(m\cdot\theta) \cdot x$

- $R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$

- $\theta_i = 10000^{(-2i/d)}, i=0,\ldots,d/2-1$

RoPE的关键性质:

相对位置编码

$\langle f(q,m), f(k,n)\rangle = g(q,k,m-n)$
内积只取决于相对位置差m-n

长距离衰减

θ越小的维度 → 区分更远距离
自然的远距离衰减特性

长度外推

RoPE + NTK-aware插值
可从4K训练扩展到128K+推理

关键要点: RoPE将位置编码为旋转角度，内积自然保持相对位置, 支持长度外推, 几乎所有现代LLM标配

RoPE: 数学推导

将d维向量分成d/2对:

$x = [x_1, x_2, x_3, x_4, \ldots, x_{d-1}, x_d]$

├─对1─┤ ├─对2─┤ ├──对d/2──┤

每对独立旋转:

$$\begin{bmatrix} x'_{2i-1} \\ x'_{2i} \end{bmatrix} = \begin{bmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{bmatrix} \begin{bmatrix} x_{2i-1} \\ x_{2i} \end{bmatrix}$$

其中 $\theta_i = 10000^{(-2i/d)}$

低维对: θ大 → 高频旋转 → 区分近距离

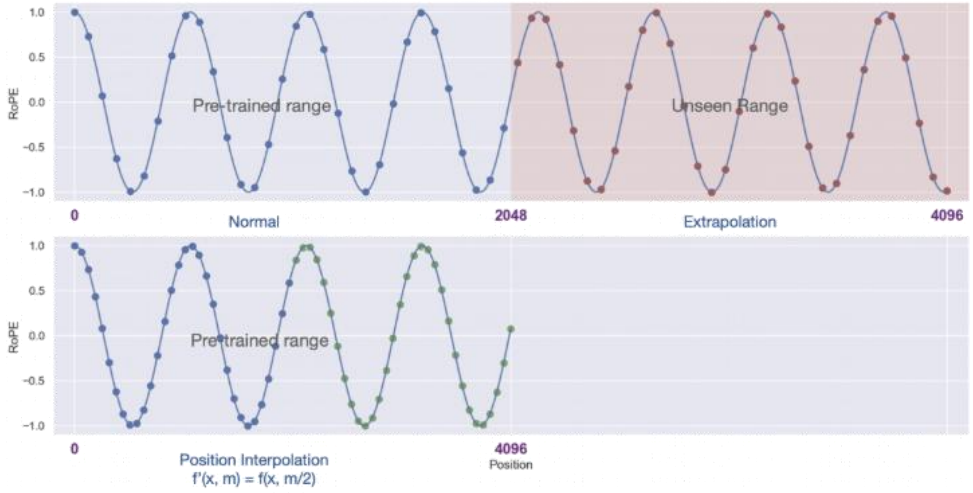
高维对: θ小 → 低频旋转 → 区分远距离

一般形式 (稀疏旋转矩阵):

$$R(m) = \begin{bmatrix} \cos(m\theta_1) & -\sin(m\theta_1) & 0 & 0 & \cdots \\ \sin(m\theta_1) & \cos(m\theta_1) & 0 & 0 & \cdots \\ 0 & 0 & \cos(m\theta_2) & -\sin(m\theta_2) & \cdots \\ 0 & 0 & \sin(m\theta_2) & \cos(m\theta_2) & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & 0 & 0 & \cos \quad -\sin \end{bmatrix}$$

- **验证相对位置性质:**
- $\langle R(m)\cdot q, R(n)\cdot k\rangle$
- $= q^T R(m)^T R(n)\, k$
- $= q^T R(n-m)\, k$ (旋转矩阵性质)
- → 只依赖相对位置差 (n-m) √

- **复数视角:**
- $f(x,m) = x \cdot e^{im\theta}$
- 两个复向量内积的相位角 = 相位差(m-n)



$m \in [0, 2048 * 2), \quad \theta'_i = \theta_i$ ✗

$m \in [0, 2048 * 2), \quad \theta'_i = \theta_i/2$ ✓

关键要点: RoPE的数学优雅: 稀疏旋转矩阵, 计算开销极小, 内积天然保持相对位置, 理论与实践完美结合

代码: RoPE Implementation

```
def precompute_freqs(d, max_len, base=10000):
    """预计算旋转频率"""
    freqs = 1.0 / (base ** (
        torch.arange(0, d, 2).float() / d
    ))                          # ★ θ_i = 10000^(−2i/d)
    t = torch.arange(max_len)
    freqs = torch.outer(t, freqs)  # (max_len, d/2)
    return torch.polar(
        torch.ones_like(freqs), freqs
    )  # ★ e^{i·m·θ} 复数形式

def apply_rope(x, freqs):
    """对Q或K应用RoPE"""
    # x: (B, n, h, d) → 复数视图
    x_complex = torch.view_as_complex(
        x.float().reshape(*x.shape[:−1], −1, 2)
    )                           # ★ 每2维→1个复数
    # 旋转: 复数乘法 = 2D旋转
    x_rotated = x_complex * freqs  # ★ 核心!
    return torch.view_as_real(
        x_rotated
    ).flatten(−2)               # 恢复实数
```

- **代码解读:**

- **precompute_freqs:**
- θ_i = 10000^(-2i/d)
- 预计算 e^{i·m·θ} 复数表
- 只需计算一次, 缓存复用

- **apply_rope:**
- 将实数向量转为复数
- 每2个实数 → 1个复数
- 复数乘法 = 2D旋转
- 这就是RoPE的全部!

- **效率:**
- 只需d/2次复数乘法
- 额外FLOPs ≈ 0 (相比MHA)

关键要点: RoPE实现仅需3步: 预计算频率 → 实数转复数 → 复数乘法(=旋转), 计算开销可忽略

KV缓存: 推理的内存瓶颈

问题: 自回归解码时, 每生成一个新token都要重新计算所有历史token的K和V，太浪费!

- **自回归推理的计算浪费:**
- 生成token t时, 需要计算:
- Attention(q , [k_1,...,k], [v_1,...,v])
- 但k_1,...,k $_{-1}$ 和 v_1,...,v $_{-1}$ 已经算过!

- **KV Cache解决方案:**
- 缓存所有已计算的K和V向量
- 每步只需计算新token的Q, K, V
- 用新的Q和缓存的全部K, V做attention

- **Prefill vs Decode:**
- Prefill: 并行处理prompt, 填充KV缓存
- Decode: 逐token生成, 每步追加KV缓存

KV Cache工作流程:

Step 1 (Prefill)

输入: "Hello world"
K_cache = [k_1, k_2]
V_cache = [v_1, v_2]

Step 2 (Decode)

新token: "!"
K_cache = [k_1, k_2, k_3]
V_cache = [v_1, v_2, v_3]

Step 3 (Decode)

新token: " How"
K_cache = [k_1, k_2, k_3, k_4]
V_cache = [v_1, v_2, v_3, v_4]

关键要点: KV Cache用空间换时间: 缓存历史K/V避免重复计算，但缓存大小随序列长度线性增长, 是推理的内存瓶颈

KV缓存: 算笔账

系统问题: KV缓存到底需要多少显存? 能服务多少用户?

KV Cache Size = 2 × n_layers × n_kv_heads × d_head × seq_len × batch_size × bytes_per_param

↑K和V ↑层数 ↑KV头数 ↑头维度 ↑序列长 ↑批大小 ↑精度(fp16=2)

模型	Layers	KV Heads	d_head	bs=1,n=4K	bs=16,n=4K	模型权重
LLaMA-2-7B	32	32 (MHA)	128	1 GB	16 GB	14 GB
LLaMA-2-13B	40	40 (MHA)	128	1.6 GB	25 GB	26 GB
LLaMA-2-70B	80	**8 (GQA)**	128	2.5 GB	40 GB	140 GB
若70B用MHA	80	64 (MHA)	128	10 GB	**160 GB!**	140 GB

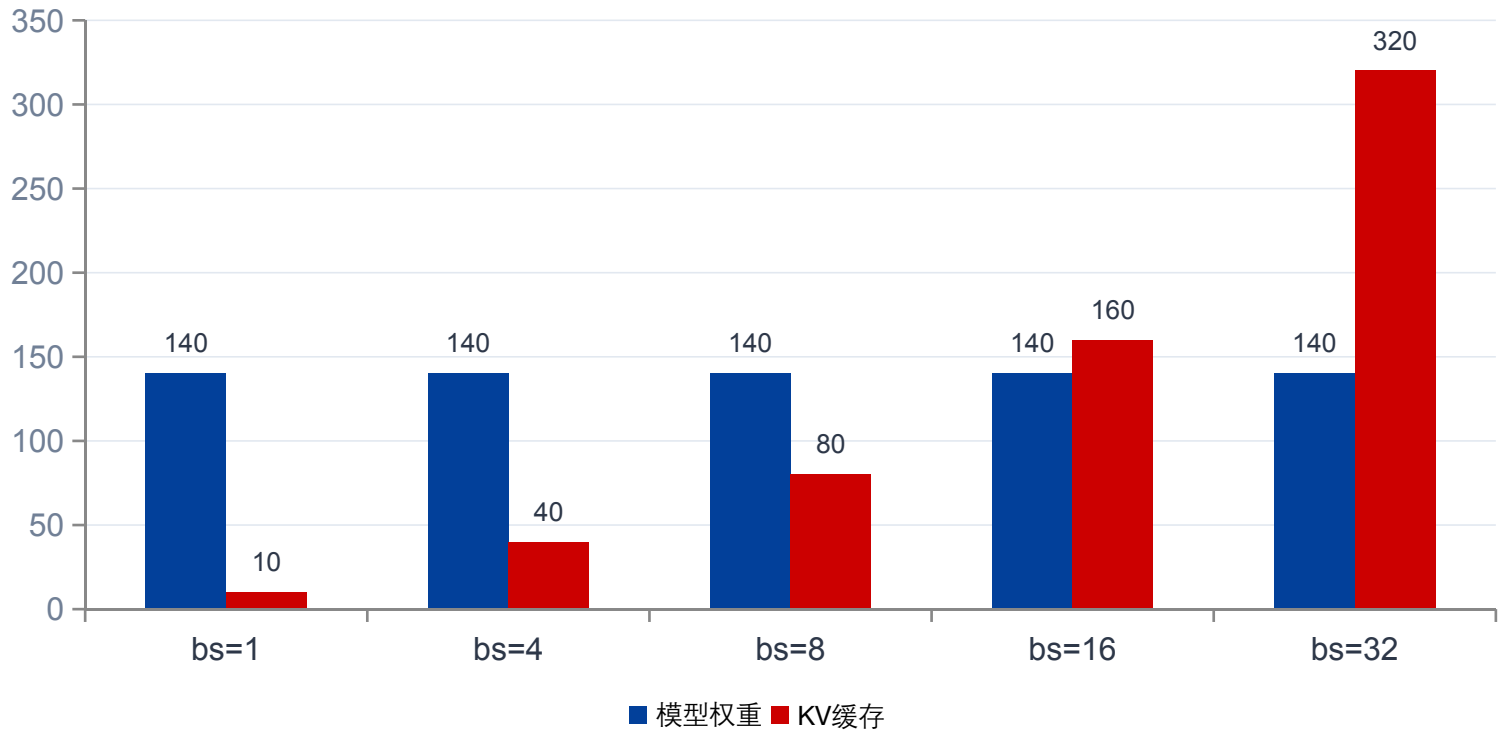
70B MHA: bs=16, n=4K → KV缓存160GB, 超过模型权重(140GB)! 需要2块A100!

70B GQA: 同样设置只需40GB KV缓存，GQA将KV头从64减到8, 缓存缩小8倍

关键要点: KV缓存是推理显存的头号消耗者，对大模型和长序列, 缓存可能比模型权重还大, 这就是GQA的动机

KV缓存 vs 模型权重: 谁更占显存?

LLaMA-2-70B (MHA, seq=4096, fp16):



关键观察:

- bs=8时, KV缓存=80GB
  已与权重(140GB)可比
- bs=16时, KV缓存=160GB
  超过模型权重!
- bs=32时, KV缓存=320GB
  需要4块A100-80GB
  仅用于存KV缓存

关键要点: batch越大, KV缓存增长越快, 这直接限制了LLM serving的吞吐量, 是系统优化的核心目标

MHA → MQA → GQA: KV缓存优化

解决方案: 减少KV头数量，多个Query头共享同一组KV, 缓存大幅缩小

MHA

N个Q头, N个KV头
每头独立的K和V
KV缓存最大

Q heads: 32 KV heads: 32

KV Cache: 100%

MQA

N个Q头, 1个KV头
所有Q共享1组KV
KV缓存最小但质量降

Q heads: 32 KV heads: 1

KV Cache: 3.1%

GQA

N个Q头, G个KV头
G组Q共享对应KV
平衡缓存与质量

Q heads: 32 KV heads: 8

KV Cache: 25%

方法	KV Cache	质量	推理速度	代表模型
MHA	100%	最优	基准	GPT-3, LLaMA-1
MQA	1/N (~3%)	略降	最快	PaLM, Falcon
GQA	**G/N (~25%)**	**≈MHA**	**快**	**LLaMA-2/3, Mistral**

关键要点: GQA是当前最优平衡: KV缓存减少75%, 质量几乎不损失，LLaMA-2-70B因此节省120GB KV缓存

训练 vs 推理: 系统挑战对比

维度	训练	推理
核心目标	吞吐量 (tokens/sec)	延迟 + 吞吐量平衡
批大小	大 (数千~数万)	动态 (1~数百)
计算类型	前向+反向, 计算密集	前向only, 访存密集(decode)
内存瓶颈	激活值+梯度+优化器状态	KV缓存 (随序列增长)
并行策略	数据/模型/Pipeline并行	Tensor并行 + 连续批处理
精度	混合精度 (bf16/fp32)	INT8/INT4量化
通信	梯度同步 (AllReduce)	KV缓存分发 (MoE)
频率	一次性 (数周~数月)	持续 (7×24, 数年)
成本占比	~6%	**~94%**

关键要点: 训练是一次性工程, 推理是持续运维, 后续章节将分别深入训练并行策略和推理优化技术

代码: 带KV Cache的推理循环

```
@torch.no_grad()
def generate(model, prompt_ids, max_new=100):
    """自回归生成 + KV Cache"""
    kv_cache = None

    # ★ Phase 1: Prefill (并行处理prompt)
    logits, kv_cache = model(
        prompt_ids,              # 全部prompt
        kv_cache=None            # 首次无缓存
    )  # kv_cache: 每层存(K, V)

    next_id = logits[:, -1].argmax(-1, keepdim=True)
    output_ids = [next_id]

    # ★ Phase 2: Decode (逐token生成)
    for _ in range(max_new - 1):
        logits, kv_cache = model(
            next_id,             # ★ 只传新token!
            kv_cache=kv_cache    # ★ 复用缓存
        )
        next_id = logits[:, -1].argmax(-1, keepdim=True)
        if next_id.item() == EOS_ID:
            break
        output_ids.append(next_id)

    return torch.cat(output_ids, dim=-1)
```

- **代码解读:**

- **Prefill (L6-10):**
- 一次前向处理全部prompt
- 计算密集: $O(n^2d)$计算量
- 生成KV Cache供后续使用

- **Decode (L15-23):**
- 每次只传入1个新token
- 访存密集: 读取全部KV Cache
- KV Cache持续增长

- **关键优化点:**
- L18: 只传新token (not全序列)
- L19: 复用已有KV Cache
- → Decode每步 $O(nd)$而非$O(n^2d)$

关键要点: KV Cache让decode从$O(n^2)$降到$O(n)$，但代价是线性增长的内存占用，这就是推理优化的核心矛盾

课程总结: 3个核心要点

01

$O(n^2)$是核心挑战

自注意力的二次复杂度是
Transformer所有系统问题的根源:
长序列计算爆炸, KV缓存线性增长,
FlashAttention和稀疏注意力都是
在对抗这个$O(n^2)$

02

每个改进源于系统瓶颈

RoPE → 解决长度泛化
GQA → 解决KV缓存过大
SwiGLU → 解决FFN表达力
MoE → 解决参数-计算耦合
不是随意改进, 是被瓶颈逼出来的

03

推理是更大的系统问题

训练一次, 推理永远:
成本占比94%, 推理模型加剧
推理 = Prefill(计算密集)
+ Decode(访存密集)
两阶段需要完全不同的优化

关键要点: 理解Transformer的系统特性, 是优化训练、推理、部署的前提, 后续章节将逐一攻克这些系统挑战